Universidade Federal de Alfenas - UNIFAL-MG
Bacharelado em Ciência da Computação
Prof. Luiz Eduardo da Silva

Trabalho de Compiladores

Construção da Árvore Sintática e Geração de Código

Objetivo

Essa prática pode ser dividade em duas etapas: (1) objetivo da primeira etapa do projeto do compilador é construir a árvore sintática para o programa em linguagem Simples. (2) Em sequência, a partir da árvore gerada, através do percurso na Árvore Sintática (AS) construída na etapa anterior, gerar o código MVS correspondente.

Problema

A análise sintática é a fase da compilação responsável por verificar se a sequência de tokens, retornados pelo analisador léxico, segue as regras sintáticas da linguagem de programação. Como resultado final da etapa de análise sintática pode-se construir uma estrutura de dados denominada árvore sintática que é uma simplificação da árvore de derivação e que representa a estrutura do programa. A árvore sintática possibilita que o mesmo programa possa ser percorrido várias vezes para realização de diversas tarefas como verificação de tipo, geração de código, etc.

Descrição

Primeira Etapa

1. Nessa primeira etapa da construção do compilador para linguagem Simples, você deverá, a partir do programa fonte, construir a árvore sintática conforme ilustrado nas Figuras 1, 2 e 3.

Programa 1: Programa para calcular o dobro

```
programa dobro
  inteiro n
inicio
  leia n
  escreva 2 * n
fimprograma
```

2. Para a implementação desse trabalho sugere-se modificar o projeto do compilador (parte 3) para gerar a árvore sintática, ao invés de gerar o código MVS das estruturas do programa.

Programa 2: Programa para calcular o fatorial

```
programa fatorial
  inteiro n fat
inicio
```

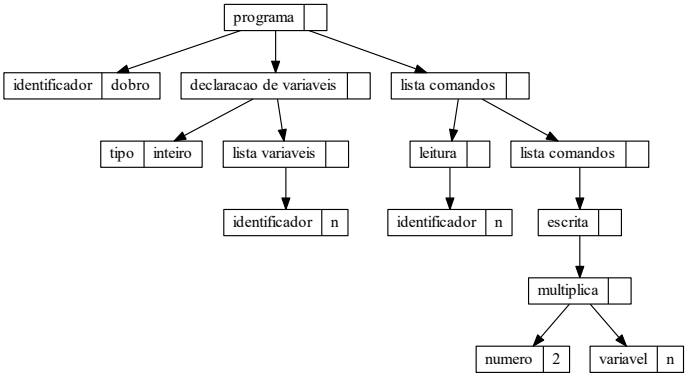

Figura 1: Árvore sintática do programa para calcular o dobro

```
  leia n
  fat <- 1
  enquanto n > 0 faca
    fat <- fat * n
    n <- n - 1
  fimenquanto
  escreva fat
fimprograma
```

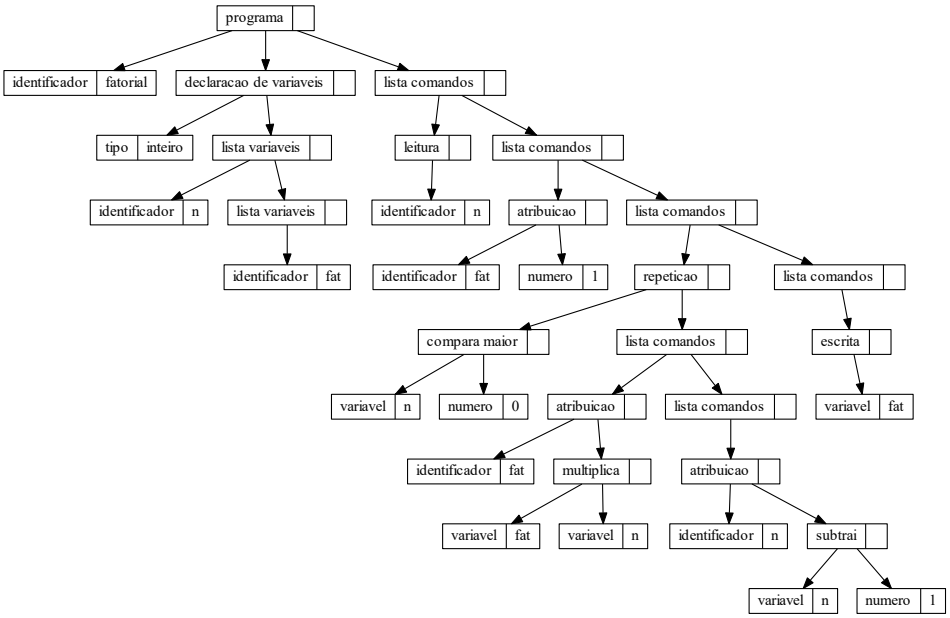

Figura 2: Árvore sintática do programa Fatorial

Programa 3: Programa para calcular o maior

```
programa maior
  inteiro a b
inicio
  leia a
  leia b
  se nao (a < b)
```

```
      entao escreva a
      senao escreva b
  fimse
fimprograma
```

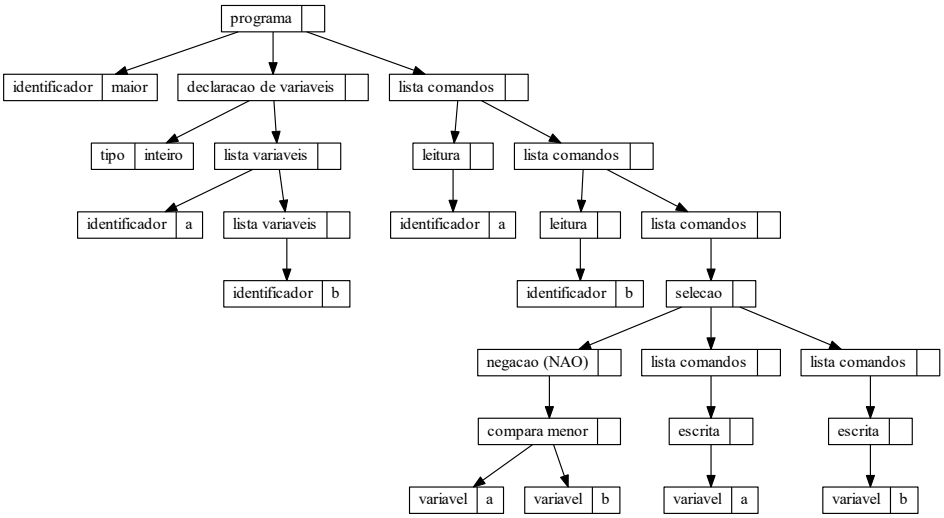

Figura 3: Árvore sintática do programa Maior

3. Deverá ser utilizada a estrutura de árvore n-ária, conforme declarações seguintes:

```
typedef struct no *ptno;

struct no {
        int tipo;
        int valor;
        ptno filho, irmao;
};

ptno criaNo (int tipo, int valor) {
   ptno n = (ptno) malloc (sizeof (struct no));
   n->valor = valor;
   n->tipo = tipo;
   n->filho = NULL;
   n->irmao = NULL;
   return n;
}

void adicionaFilho (ptno pai, ptno filho) {
        if (filho) {
           filho->irmao = pai->filho;
           pai->filho = filho;
        }
}
```

4. A saída dessa etapa do projeto deverá ser a sequência de comandos para desenhos de gráficos no formato .DOT (graphViz). Uma descrição completa da ferramenta usada para gerar ilustrações pode ser obtida em "https://www.graphviz.org/pdf/dotguide.pdf". Por exemplo, a sequência de comandos para construção da árvore sintática do programa DOBRO é:

```
digraph {
 node [shape=record, height=.1];
 n0x600086260 [label = "programa | "];
 n0x600085e20 [label = "identificador | dobro"];
 n0x600085f40 [label = "declaracao de variaveis | "];
 n0x600085e50 [label = "tipo | inteiro"];
 n0x600085ef0 [label = "lista variaveis | "];
 n0x600085ea0 [label = "identificador | n"];
 n0x600086210 [label = "lista comandos | "];
 n0x600085fe0 [label = "leitura | "];
 n0x600085f90 [label = "identificador | n"];
 n0x6000861c0 [label = "lista comandos | "];
 n0x600086170 [label = "escrita | "];
 n0x600086120 [label = "multiplica | "];
 n0x600086030 [label = "numero | 2"];
 n0x6000860d0 [label = "variavel | n"];
 n0x600086260 -> n0x600085e20;
 n0x600086260 -> n0x600085f40;
 n0x600085f40 -> n0x600085e50;
 n0x600085f40 -> n0x600085ef0;
 n0x600085ef0 -> n0x600085ea0;
 n0x600086260 -> n0x600086210;
 n0x600086210 -> n0x600085fe0;
 n0x600085fe0 -> n0x600085f90;
 n0x600086210 -> n0x6000861c0;
 n0x6000861c0 -> n0x600086170;
 n0x600086170 -> n0x600086120;
 n0x600086120 -> n0x600086030;
 n0x600086120 -> n0x6000860d0;
}
```

5. Para compor o nome dos nós utilizou-se, nesse exemplo, o endereço hexadecimal dos ponteiros dos nós. Neste caso, n0x600086260 foi gerado com o seguinte comando:

```
 printf ("\tn%p", raiz);
```

6. Para verificar graficamente a árvore construída, pode-se usar o comando, como por exemplo, "dot -Tsvg dobro.dot -o dobro.svg". Neste exemplo, o arquivo *dobro.dot* é o arquivo texto que contém a sequência apresentada anteriormente. O gráfico gerado pode ser aberto num navegador, pois o formato SVG (Scalable Vector Graphic) é um formato de imagens vetoriais que podem ser utilizadas em páginas HTML.

Segunda Etapa

1. Nessa segunda etapa da construção do compilador para linguagem Simples, você deverá, a partir da árvore sintática construída na etapa anterior, gerar o código correspondente para MVS do programa fonte representado na árvore sintática. As Figuras 4 e 5 representam trechos da árvore sintática referente a um programa simples qualquer.

2. Para o percurso na árvore sugere-se a implementação da rotina recursiva *geracod*, que recebe como parâmetro um ponteiro de nó da árvore sintática. Para cada tipo de nó deverão ser feitas as traduções e as chamadas recursivas correspondentes para aquele trecho da árvore. Por exemplo, para os trechos da AS das Figuras 4 e 5, a rotina *geracod* que gera código MVS (máquina virtual simples) tem o seguinte tratamento:

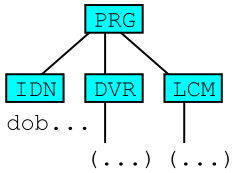

Figura 4: Trecho da árvore referente a estrutura principal do programa

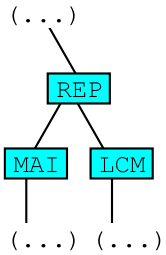

Figura 5: Trecho da árvore referente a repetição

```
void geracod (ptno p) {
   if (p == NULL) return;
   ptno p1, p2, p3;
   switch (p->tipo) {
      case PRG:
         p1 = p->filho;
         p2 = p1->irmao;
         p3 = p2->irmao;
         printf ("\tINPP\n");
         NUMLVAR = 0;
         geracod (p2);
         geracod (p3);
         printf ("\tFIMP\n");
         break;
        (...)
       case REP:
         p1 = p->filho;
         p2 = p1->irmao;
         printf ("L%d\tNADA\n", ++ROTULO);
         empilha (ROTULO);
         geracod (p1);
         printf ("\tDSVF\tL%d\n",++ROTULO);
         empilha (ROTULO);
         geracod (p2);
         aux = desempilha();
         printf ("\tDSVS\tL%d\n", desempilha());
         printf ("L%d\tNADA\n", aux);
         break;
         (...)
   }
}
```

3. Observe na Figura 4 que o nó do tipo PRG (Programa) tem três filhos: o nome do programa para a qual não é necessária nenhuma tradução, a declaração de variáveis (DVR) e a lista de comandos do corpo principal do programa (LCM). As chamadas

recursivas para tradução de DVR e LCM são colocadas entre a geração dos código INPP e FIMP da MVS para o início e o fim do programa.

4. O nó do tipo REP (repetição) da Figura 5 tem dois filhos: uma expressão (condição da repetição, que no exemplo é a comparação maior, MAI) e a lista de comandos (LCM) da repetição. A geração dos rótulos e os comandos de desvio (DSVS e DSVF) estão colocados nos locais apropriados para a tradução.

5. A saída dessa etapa do projeto deverá ser o código MVS correspondente ao programa fonte em linguagem Simples.

6. Como uma etapa adicional, pode-se percorrer a árvore novamente para realizar as verificações de tipo.

Entrega

1. Incluir um comentário no cabeçalho de cada programa fonte com o seguinte formato:

```
/*+-------------------------------------------------------------------------
  |            UNIFAL - Universidade Federal de Alfenas.
  |              BACHARELADO EM CIENCIA DA COMPUTACAO.
  | Trabalho..: Construcao Arvore Sintatica e Geracao de Codigo
  | Disciplina: Teoria de Linguagens e Compiladores
  | Professor.: Luiz Eduardo da Silva
  | Aluno.....: Fulano da Silva
  | Data......: 99/99/9999
  +-------------------------------------------------------------------------*/
```

2. A pasta com o projeto deverá incluir o seguinte arquivo Makefile:

```
simples : utils.c lexico.l sintatico.y;
        @flex -o lexico.c lexico.l
        @bison -v -d sintatico.y -o sintatico.c
        @gcc lexico.c sintatico.c -o simples
limpa : ;
        @echo "limpando..."
        @rm lexico.c sintatico.c sintatico.h sintatico.output simples
```

3. O compilador deverá ter o nome "simples"(linux) ou "simples.exe"(windows) e ser chamado através da seguinte linha de comando:

```
./simples <nomeprograma>[.simples]
```

   A partir deste comando deverão ser gerados os arquivos ¡nomeprograma¿.dot, ¡nomeprograma¿.svg e ¡nomeprograma¿.mvs.

4. Enviar num arquivo único (.ZIP), a pasta do projeto com somente os arquivos fontes (lexico.l, sintatico.y, tree.c, tree.h, utils.c e makefile), através do Envio de Arquivo do MOODLE.